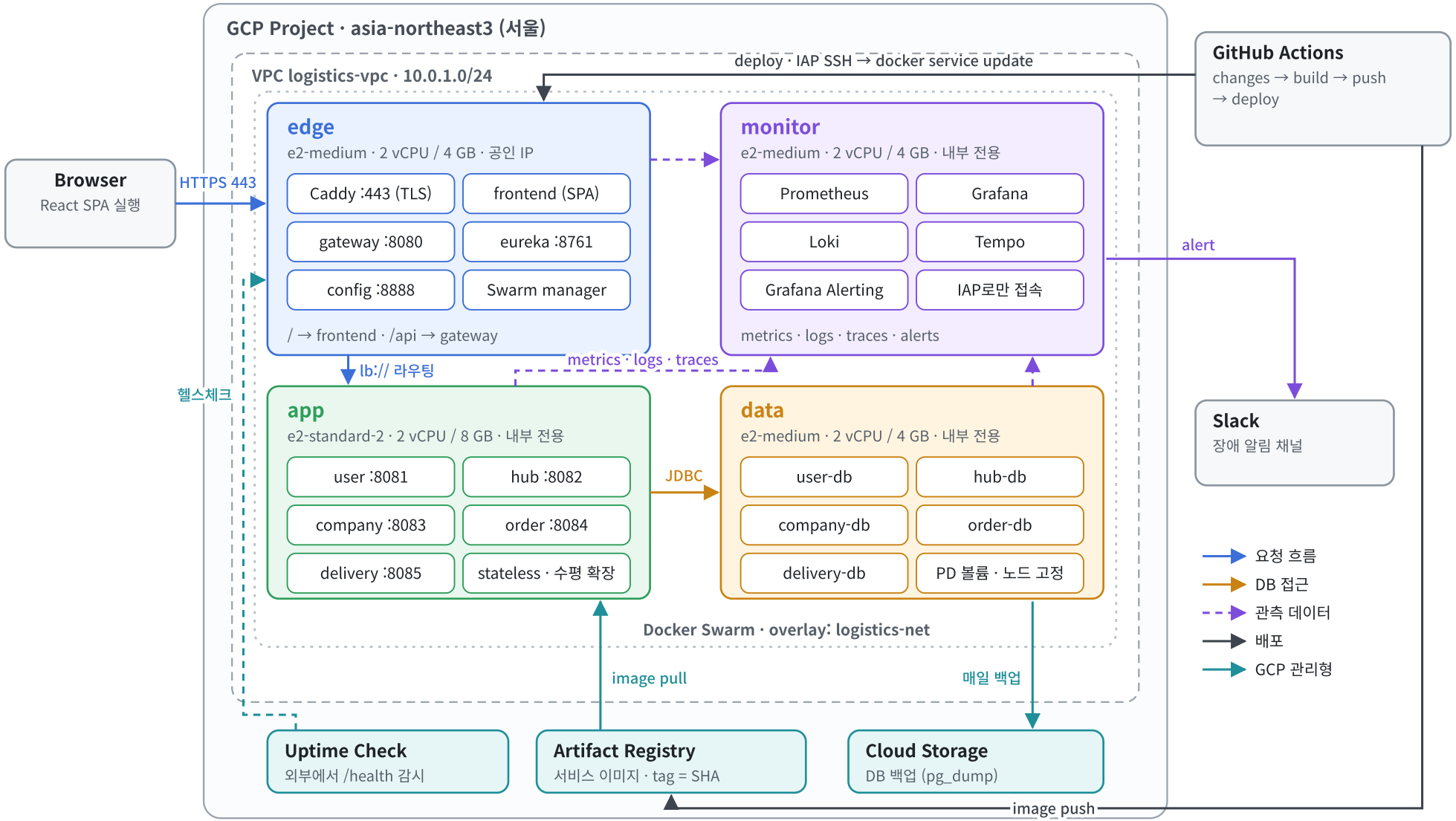
01 인프라 전체 구성

GCP Compute Engine 4대 + Docker Swarm · 서비스별 DB 분리 · 프론트는 같은 도메인으로 서빙



왜 4개 노드로 나눴나

상태가 있는 DB(data)와 상태가 없는 앱(app)을 분리해 앱을 재배포하거나 앱에서 메모리가 터져도 DB는 영향이 없게 했습니다. 감시하는 쪽(monitor)은 감시받는 쪽과 다른 노드에 둡니다.

프론트엔드

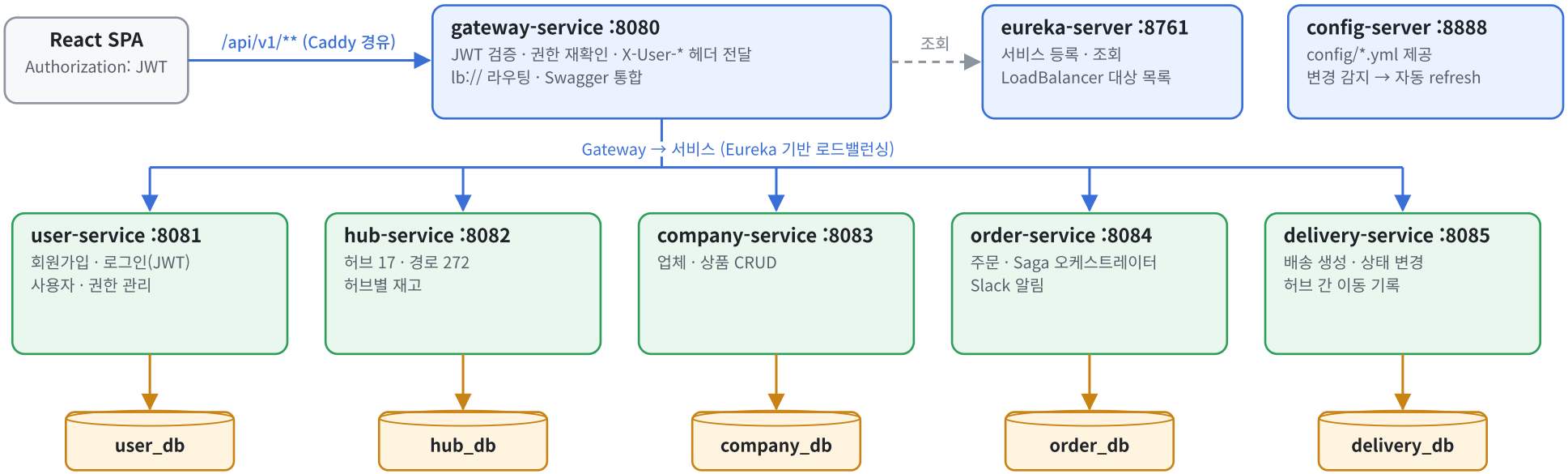
빌드된 React 정적 파일을 edge의 작은 컨테이너가 서빙합니다. Caddy가 / 는 frontend, /api 는 gateway로 나눠 같은 도메인이 되므로 CORS 설정이 필요 없고 노드도 늘지 않습니다.

외부 노출 최소화

공인 IP는 edge 하나, 열린 포트는 443뿐입니다. DB와 앱 포트는 overlay 네트워크 안에서만 통신하고, SSH는 IAP 터널로만 접속합니다.

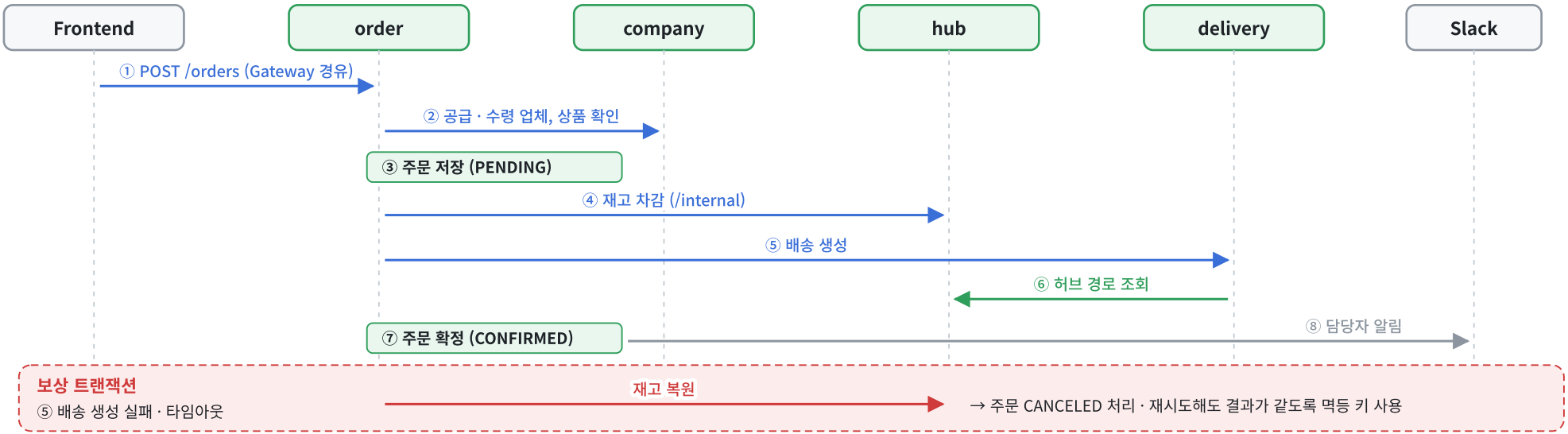
Database per Service

서비스마다 Postgres 컨테이너와 볼륨을 따로 둡니다. 다른 서비스 DB 접근은 금지하고, 데이터가 필요하면 OpenFeign으로 그 서비스의 /internal API를 호출합니다.



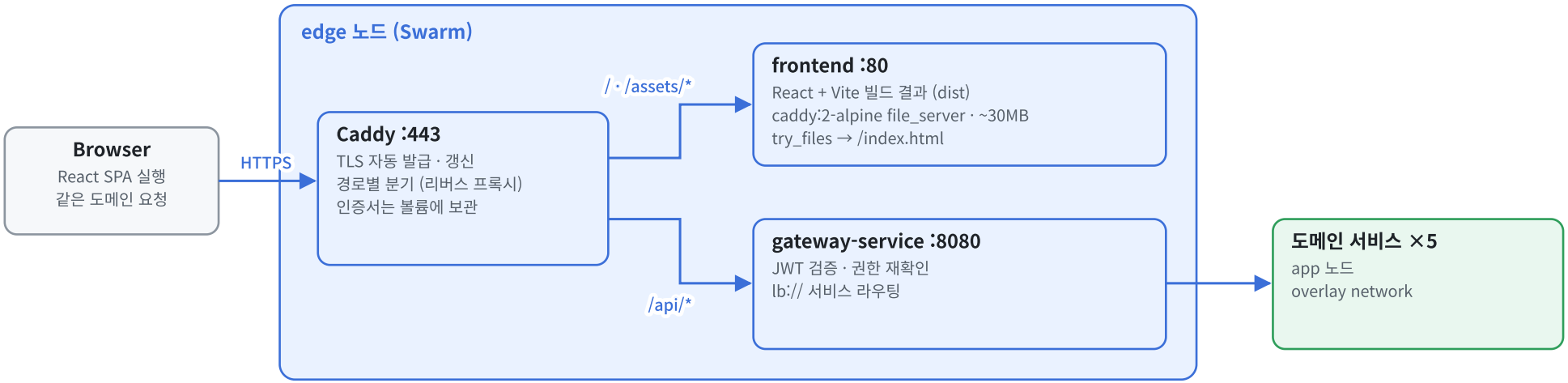
모든 서비스: 기동 시 config-server에서 설정 수신 → eureka 등록 · 서비스 간 호출은 OpenFeign /internal/** (Gateway 미경유)

주문 생성 Saga (Orchestration) — 설계안



03 프론트엔드 구성

React SPA를 edge 노드에서 같은 도메인으로 서빙 · 독립 배포



프론트 배포 = frontend 컨테이너만 교체 · Caddy(진입점)와 백엔드는 그대로

Caddyfile (edge)

```
logistics.example.com {
  handle /api/* {
    reverse_proxy gateway-service:8080
  }
  handle {
    reverse_proxy frontend:80
  }
}
```

같은 도메인 → **CORS 설정 불필요**. 진입점(Caddy)과 프론트를 분리해서 프론트 배포 시 TLS · 라우팅이 흔들리지 않습니다.

레포 · 이미지

```
frontend/
  src/  vite.config.ts
  Dockerfile        # multi-stage
    node:20-alpine → npm ci · build
    caddy:2-alpine → /srv (dist)
.github/workflows/deploy.yml
  frontend/** → frontend 이미지만
```

로컬: Vite proxy로 `/api → localhost:8080` (docker compose). 로컬도 CORS 없이 개발.

API 계약 · 인증

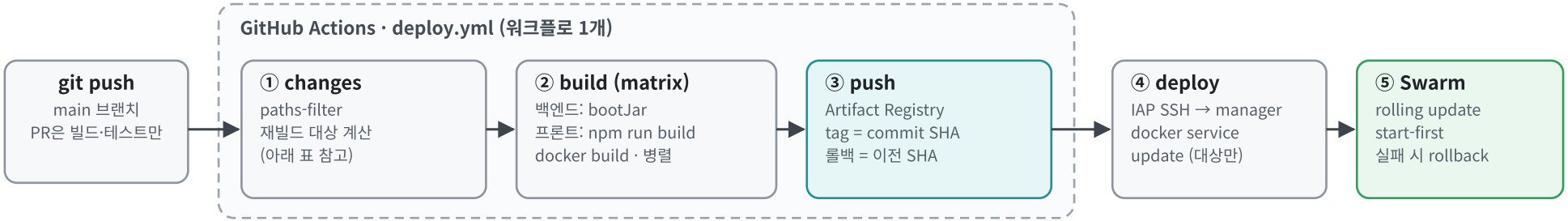
항목	방식
API 타입	Gateway 통합 Swagger → openapi-typescript / orval로 TS 타입 생성. 백엔드 응답이 바뀌면 프론트 빌드에서 바로 감지
인증	로그인 응답 JWT → `Authorization` 헤더 (지금 Gateway 방식 그대로)
개선 과제	localStorage는 XSS에 노출 → httpOnly 쿠키로 전환 (같은 도메인이라 쉬움)

화면 범위 (MVP) — MSA 흐름을 눈으로 보여주는 시연용

화면	하는 일	호출하는 서비스	권한
로그인 · 회원가입	JWT 발급, 역할에 따라 메뉴 분기	user	전체
주문 생성	업체 · 상품 선택 → 주문 → 결과(확정 / 보상 후 취소) 표시 · Saga 시연	company · order (→ hub · delivery)	업체 담당자
주문 · 배송 조회	배송 상태, 허브 간 이동 기록 (예상 vs 실제)	order · delivery	전체
관리	허브 · 업체 · 사용자 목록, 배송 담당자 지정	hub · company · user	MASTER · 허브 관리자

04 배포 파이프라인 · 무중단 배포

바뀐 서비스만 빌드 · 배포하고, 새 버전이 정상일 때만 기존 버전을 내립니다



변경 범위 → 재빌드 대상

바뀐 경로	재빌드 · 재배포	이유
backend/order-service/**	**order 1개**	독립 배포
backend/common/**	**비즈니스 서비스 5개**	common.jar가 각 서비스 jar 안에 포함됨
build.gradle · gradle/**	**전체 8개**	버전 · 플러그인 공통
frontend/**	**frontend 1개**	정적 파일 이미지만 교체 · 백엔드 · Caddy 재시작 없음
config/**	**없음**	파일 동기화 → 자동 refresh

DB 스키마 변경 — Flyway + expand / contract

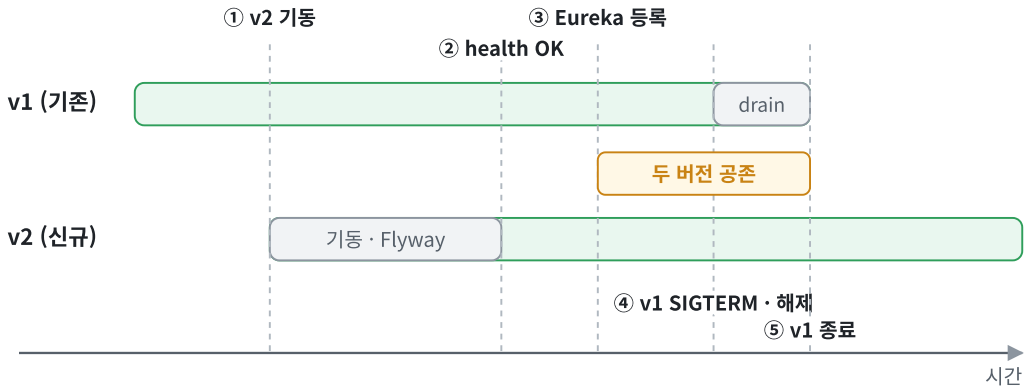
1. expand
컬럼 추가 (nullable)
v1 · v2 모두 동작

2. 코드 배포
v2가 새 컬럼 사용
rolling update

3. contract
옛 컬럼 삭제
다음 배포에서

API도 같은 원칙: 필드는 추가만 하고, 호출받는 쪽을 먼저 배포합니다.

무중단 교체 (start-first rolling update)



health 실패 시 → v1을 내리지 않고 자동 rollback (failure_action: rollback)

graceful shutdown + Eureka 해제 후 종료 → 진행 중 요청 유실 방지. 검증: 배포 중 k6로 요청을 계속 보내 **에러 0건**을 Grafana로 확인.

한 서비스만 배포하기 전 체크리스트

① 스키마 변경?
Flyway expand 마이그레이션이 포함됐는지. 컬럼 삭제 · 이름 변경은 다음 배포로 미룸.

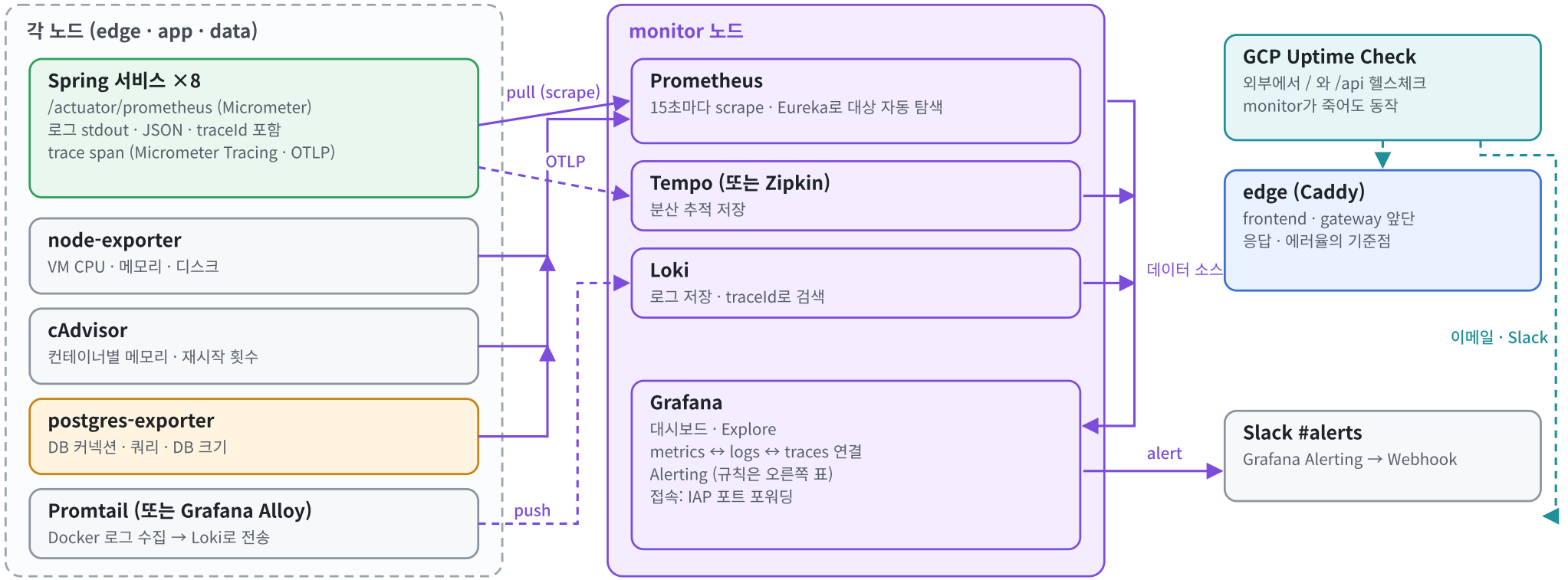
② API 변경?
호출받는 서비스를 먼저 배포. 응답 필드는 추가만, 삭제 · 이름 변경 금지.

③ common 변경?
섞여 도는 구간이 있으므로 하위 호환 유지 (예: 헤더 이름은 새 이름 · 옛 이름 둘 다 허용).

④ 배포 후 5분
Grafana에서 해당 서비스 에러율 · p95 확인. 이상하면 이전 SHA로 `docker service update` 또는 `rollback`.

05 관측성 (모니터링 · 로그 · 추적 · 알림)

Prometheus · Loki · Tempo · Grafana — 하나의 traceId로 메트릭 · 로그 · 추적을 연결



알림 규칙 (Grafana Alerting → Slack)

규칙	조건	심각도
서비스 다운	up == 0 · 1분 지속	critical
5xx 비율	> 5% · 5분	critical
응답 지연	p95 > 1s · 5분	warning
JVM 힙 · 컨테이너 메모리	> 90% (limit 대비)	warning
서킷브레이커 OPEN	Resilience4j state == open	warning
디스크	data 노드 > 80%	warning

대시보드

대시보드	보는 것
서비스 개요	서비스별 RPS · 에러율 · p95 (RED)
JVM	힙 · GC · 스레드 · HikariCP 커넥션
인프라	노드 CPU · 메모리 · 디스크, 컨테이너 재시작
주문 Saga	주문 성공 · 보상 건수, 단계별 소요시간 (trace)
배포	배포 시점 표시 + 같은 구간 에러율

06 노드 · 네트워크 · 운영 기준

사양, 방화벽, 백업, 주의점, 우선순위

노드 사양 · 메모리 예산

추정치 · docker stats로 실측 후 조정

노드	머신	컨테이너 (limit)	예상 사용
edge	e2-medium · 4GB	Caddy 64M · frontend 32M · gateway 512M · eureka 384M · config 384M · exporter 128M	≈ 1.5 GB
app	e2-standard-2 · 8GB	서비스 5개 × 512M · exporter 128M	≈ 2.7 GB order ×2 확장 여유
data	e2-medium · 4GB	Postgres 5개 × 256M (shared_buffers 32MB)	≈ 1.3 GB
monitor	e2-medium · 4GB	Prometheus 768M · Loki 512M · Tempo 512M · Grafana 256M	≈ 2.0 GB

JVM: `-XX:MaxRAMPercentage=75 -XX:+UseSerialGC` · Swarm `deploy.resources.limits.memory`로 제한. DB · 모니터링은 `node.labels.role`로 노드 고정.

백업 · 비용

항목	방법
DB 백업	매일 pg_dump (DB 5개) → Cloud Storage · 7일 보관
디스크	data 노드 PD 스냅샷 스케줄 (일 1회)
복구 리허설	백업으로 새 볼륨 복원 1회 해보고 문서화
비용	무료 체험 크레딧 사용 · 예산 알림 50% / 90% · 시연 후 VM 중지

방화벽 · 접근 제어

규칙	출발지	대상 · 포트
allow-https	0.0.0.0/0	edge · tcp 80, 443
allow-iap-ssh	35.235.240.0/20 (IAP)	전체 · tcp 22
allow-swarm	10.0.1.0/24	전체 · tcp 2377, 7946 · udp 7946, 4789
allow-internal	10.0.1.0/24	overlay 내부 서비스 · DB 포트
(기본)	그 외	인바운드 전부 차단

DB 포트는 호스트에 publish하지 않습니다. DB 클라이언트 · Grafana는 IAP 터널로 접속. 비밀값(DB 비밀번호, JWT_SECRET, Slack webhook)은 Swarm secret.

Swarm 운영 시 주의점

문제	대응
depends_on 무시됨	config client fail-fast + retry · restart_policy
Eureka에 엉뚱한 IP 등록	spring.cloud.inetutils.preferred-networks = overlay 대역
DB 컨테이너 노드 이동	placement: node.labels.role == data
종료 시 요청 유실	server.shutdown=graceful · LoadBalancer retry

진행 순서 (~10/22)

필수

① Terraform: VPC · 방화벽 · VM 4대 · 버킷 → ② Swarm 구성 · stack 배포 → ③ GitHub Actions 파이프라인 → ④ start-first · healthcheck · graceful → ⑤ Prometheus + Grafana + Slack 알림 → ⑥ Flyway → ⑦ 프론트 MVP 화면 4개

여유 있으면

JWT → httpOnly 쿠키 · Loki 로그 · Tempo 추적 · 백업 자동화 · 배포 중 k6 무중단 검증 캡처

이번엔 안 함

GKE / k8s 전환 → ADR에 다음 단계로 기록